

UE23CS352B — Software Engineering Lab (Lab 4: VibeCoding)

Prompt-Driven Game Debugging & Feature Engineering Transcript

Student Name: Krrish P Raju	SRN: PES1UG24CS708
Assigned Problem: #44 Simple Platformer	Pair-Programming Assistant: Gemini / Claude LLM

1. Executive Summary & Defect Diagnosis

Upon inspecting and playtesting the starter codebase for Repository #44 (44_simple-platformer), two critical software defects were discovered that severely degraded gameplay experience:

- **Bug 1 (Physics Tunneling):** Vertical acceleration accumulated under gravity without terminal velocity capping. When jumping off the elevated platform ($y = 400$), downward velocity exceeded 16 px/frame. Because platforms are only 14 px thick and collision was checked discretely after moving, the player's bounding box bypassed the platform entirely.
- **Bug 2 (Game Freeze on Death):** When colliding with hazards or dropping off-screen, the engine set `game_over = True`, logged a single print statement to stdout, and halted the update loop. The Pygame window froze permanently with no visual feedback, game-over UI, or restart mechanism.

Using a prompt-driven vibe coding methodology, these defects were diagnosed, isolated, and resolved, and all four tasks specified in the README were implemented in under 4 prompt iterations.

2. Prompt-Driven Pair Programming Session Transcript

PROMPT 1 (Student — Krrish P Raju):

I'm testing the Simple Platformer starter code from SETAPESU26/44_simple-platformer. While playtesting, I identified two game-breaking bugs:

1. **Collision Tunneling:** When falling from the elevated platform, vertical speed accelerates uncontrollably and the player drops straight through the 14px platforms instead of landing on them.
2. **Game Freeze on Death:** When hitting a hazard or falling into a pit, the game logs a single line to the console (`'Game over! Final score: X'`) and completely freezes in-place with no UI overlay or way to replay.

Can you explain why discrete collision detection is failing here and refactor `game_engine.py` to use continuous swept collision with terminal velocity capping, while adding a proper state machine for `PLAYING` vs `GAME_OVER`?

ASSISTANT RESPONSE 1:

Great catch! Here is the architectural analysis and fix:

1. **Why Discrete Collision Fails:** In `game_engine.py`, `player.y += player.vy` is executed before collision checks. If `player.vy` exceeds `platform.height` (14px), the player's bounding box jumps from above the platform to below it in a single frame. Pygame's `colliderect` never evaluates to `True` because the rectangles never overlap.

Fix: We implement Continuous Swept Collision Detection by testing whether the vertical interval `[old_bottom, new_bottom]` crosses the platform surface `platform.y` while horizontally aligned. Additionally, we enforce a `terminal_velocity` cap (14.0 px/frame) and snap `player.y = platform.y - player.height` with `player.vy = 0`.

2. **State Machine:** We replace the bare boolean with an explicit state model (`self.state = 'PLAYING' | 'GAME_OVER'`). When death conditions trigger, the engine transitions state cleanly instead of halting abruptly.

PROMPT 2 (Student — Krrish P Raju):

The swept collision fix worked perfectly! The player now lands cleanly every time regardless of fall height, and the game enters the `GAME_OVER` state without crashing.

Now let's complete Tasks 2 and 3 from the README:

1. **Game Over Overlay:** Build a sleek in-game overlay screen that displays 'GAME OVER', the final score, and best score.
2. **Replay & Difficulty Selection:** After Game Over, allow the user to replay by choosing a difficulty (Easy: 0.45 gravity & floaty jump, Medium: standard 0.60 gravity, Hard: 0.85 heavy gravity) or exit gracefully. Let's bind keys [1], [2], [3] for difficulty and [R] to replay.

ASSISTANT RESPONSE 2:

Here is the implementation for Tasks 2 and 3:

- **Difficulty Engine:** We define a `DIFFICULTIES` dictionary mapping each preset to custom gravity, `jump_strength`, speed, and `terminal_velocity` values. An `apply_difficulty()` method configures kinematics dynamically.
- **Game Over Rendering:** We create a semi-transparent dark backdrop overlay (alpha 210) with a styled central dialogue panel. It displays 'GAME OVER', 'Final Score', 'Best Score', and interactive keyboard shortcuts.
- **Replay State Handling:** When in `GAME_OVER` state, `handle_event()` listens for [1] Easy, [2] Medium, [3] Hard, or [R] Replay, which resets player position, zeroes score, restores 'PLAYING' state, and preserves the session high score.

PROMPT 3 (Student — Krrish P Raju):

The replay menu and difficulty options work seamlessly! Players can now select Easy, Medium, or Hard and immediately jump back into the action. Now for Task 4: Add sound feedback for jumping, reaching the goal, and dying. Rather than relying on external .wav or .mp3 files (which can cause missing asset errors or path incompatibilities), can we procedurally synthesize authentic 8-bit retro sound effects directly in Python memory using pygame.mixer?

ASSISTANT RESPONSE 3:

Procedural waveform synthesis is an elegant, zero-dependency solution! Here is how we build game/sounds.py:

- We use Python's built-in math and struct modules to generate 16-bit PCM RIFF WAV byte streams directly in memory:

1. **Jump:** Upward pitch sweep (260 Hz → 640 Hz) square wave with sharp exponential decay.
2. **Goal:** Upward arpeggio sweep (480 Hz → 960 Hz) melodic sine wave chime.
3. **Death:** Descending pitch drop (340 Hz → 75 Hz) sawtooth wave with low-frequency rumble.

- We integrate SoundManager into GameEngine so jumping, crossing the green goal line, and touching hazards trigger audio feedback instantly with graceful hardware fallback.

PROMPT 4 (Student — Krrish P Raju):

I thoroughly verified all changes! The platform collision is rock solid, the difficulty physics feel distinct, audio feedback triggers cleanly, and the game over screen allows instant replays. I have captured both 10-second before and after gameplay videos. Everything is ready for final submission.

ASSISTANT RESPONSE 4:

All requirements for Lab 4 are fully satisfied! You have resolved both critical bugs and delivered all 4 tasks specified in the lab assignment. Deliverables (before/after videos, updated codebase, and this chat history) are ready to be pushed to your GitHub repository under Lab-4/.

3. Deliverables & Verification Matrix

Task / Deliverable	Original Starter Behavior	Resolved Implementation
Task 1: Collision	Discrete check; player tunneled through platforms during high-speed falls.	Continuous swept collision + terminal velocity cap (14 px/frame); 100% reliable landing.
Task 2: Game Over	Stdout print only; game froze indefinitely with zero graphical UI.	State machine (GAME_OVER) with semi-transparent overlay, final score, and best score.
Task 3: Replay & Diff	No replay capability without restarting Python interpreter.	Dynamic difficulty presets (Easy: 0.45, Med: 0.60, Hard: 0.85) with keyboard shortcuts.
Task 4: Sound FX	No audio feedback existed.	Procedural 8-bit WAV synthesis for Jump, Goal reached, and Death/Game Over.